Factory Method Design Pattern — Complete Guide (Beginner → Advanced, Python 3)

Every code example in this guide was actually run to confirm it behaves exactly as described.

1. The Big Idea in One Line

Factory Method means: **don't call a constructor directly — call a method whose whole job is to decide which class to build, and hand you back the result.**

Think of a pizza chain. Every store follows the same overall process — take the order, prepare it, bake it, box it. But the *specific* pizza that gets prepared depends on which store you're at: an NYC franchise prepares an NY-style pizza, a Chicago franchise prepares a deep-dish. The overall process (order → prepare → bake → box) never changes, but one step — "make the actual pizza" — is deliberately left open for each store to decide. That one open step is the factory method.

2. Formal Definition (the one interviewers expect)

> Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.

Two ideas packed into that sentence:

1. There's a common interface/method for "create the object" — callers use that method, never `SomeConcreteClass()` directly.
2. The *decision* of exactly which concrete class gets built is pushed down into subclasses (or, in more Pythonic variants you'll see below, into a registry or parameter) — not hard-coded in the calling code.

3. Why Do We Even Need This?

Imagine code that directly does this:

```python
def plan_delivery(kind):
    if kind == "road":
        transport = Truck()
    elif kind == "sea":
        transport = Ship()
    return transport.deliver()
```

Every time you add a new transport type (say, `Plane`), you have to come back and edit this function, adding another `elif`. This function is directly coupled to every concrete class it might create — it has to know about `Truck`, `Ship`, and everything else, forever.

Factory Method flips this: the "which class do I build" decision moves into something replaceable (a subclass, in the classic version) so the calling code never needs to change when a new type is added. This is the textbook example interviewers use to test whether you understand the **Open/Closed Principle** from SOLID — *open for extension, closed for modification.*

4. The Structure

(This is the diagram shown above in chat.)

- **Product** (`Transport`) — the common interface every object the factory method creates must follow.
- **ConcreteProduct** (`Truck`) — an actual implementation of that interface.
- **Creator** (`Logistics`) — declares the factory method (`create_transport()`), and usually also contains real business logic (`plan_delivery()`) that *uses* whatever the factory method returns, without caring which concrete class it actually got.
- **ConcreteCreator** (`RoadLogistics`) — overrides the factory method to return a specific `ConcreteProduct`.

5. The Basic Implementation

python

```python
from abc import ABC, abstractmethod

# --- Product interface ---
class Transport(ABC):
    @abstractmethod
    def deliver(self):
        pass

# --- Concrete Products ---
class Truck(Transport):
    def deliver(self):
        return "Delivering by land in a box"

class Ship(Transport):
    def deliver(self):
        return "Delivering by sea in a container"

# --- Creator (abstract) ---
class Logistics(ABC):
    @abstractmethod
    def create_transport(self) -> Transport:
        """The factory method."""
        pass

    def plan_delivery(self):
        # business logic that only knows about the Transport interface,
        # never about Truck or Ship directly
        transport = self.create_transport()
        return transport.deliver()

# --- Concrete Creators ---
class RoadLogistics(Logistics):
    def create_transport(self) -> Transport:
        return Truck()

class SeaLogistics(Logistics):
    def create_transport(self) -> Transport:
        return Ship()


def client_code(logistics: Logistics):
    print(logistics.plan_delivery())

client_code(RoadLogistics())  # Delivering by land in a box
client_code(SeaLogistics())   # Delivering by sea in a container
```

Notice `client_code` never mentions `Truck` or `Ship` anywhere. It only knows about `Logistics` and, indirectly, `Transport`. To add air delivery, you write `Plane` and `AirLogistics` — you never touch `client_code` or `Logistics` at all. That's Open/Closed in action.

⚠️ Gotcha: forgetting to override the factory method

```python
class BrokenLogistics(Logistics):
    pass  # forgot to implement create_transport

BrokenLogistics()
# TypeError: Can't instantiate abstract class BrokenLogistics
# without an implementation for abstract method 'create_transport'
```

(Verified.) Because `Logistics` uses `ABC` + `@abstractmethod`, Python refuses to let you instantiate any subclass that hasn't overridden `create_transport`. This is a genuine safety net — without `ABC`, a forgotten override would silently do nothing or blow up much later with a confusing error, instead of failing loudly at the moment you try to create the broken subclass. Same applies to `Logistics()` itself — you can't instantiate the abstract Creator directly either (verified — same `TypeError`).

6. "Simple Factory" vs. Factory Method vs. Abstract Factory

This is one of the most common interview trip-ups. Know the difference cold.

Simple Factory (not an official GoF pattern)

One function or static method with an if/elif chain that returns different objects based on a parameter:

```python
class SimpleTransportFactory:
    @staticmethod
    def create_transport(kind: str) -> Transport:
        if kind == "road":
            return Truck()
        elif kind == "sea":
            return Ship()
        raise ValueError(f"Unknown transport kind: {kind}")
```

This is what most beginners actually mean when they say "factory." It's easy to write, but every new type means editing this method's if/elif chain — it violates Open/Closed. It's also not one of the 23 official GoF patterns; it's just a common, practical idiom people label "factory."

Factory Method (the real GoF pattern — Sections 4–5 above)

Creation is deferred to **subclasses** through method overriding (polymorphism). Adding a new product means adding a new `ConcreteCreator` + `ConcreteProduct` pair — zero existing code touched. Uses inheritance.

Abstract Factory (a related but different pattern)

An object with **several** factory methods bundled together, used to create **families of related objects that must be used together**.

python

```python
class Button(ABC):
    @abstractmethod
    def render(self): pass

class Checkbox(ABC):
    @abstractmethod
    def render(self): pass

class WindowsButton(Button):
    def render(self): return "Windows-style button"

class WindowsCheckbox(Checkbox):
    def render(self): return "Windows-style checkbox"

class MacButton(Button):
    def render(self): return "Mac-style button"

class MacCheckbox(Checkbox):
    def render(self): return "Mac-style checkbox"

class GUIFactory(ABC):
    @abstractmethod
    def create_button(self) -> Button: pass
    @abstractmethod
    def create_checkbox(self) -> Checkbox: pass

class WindowsFactory(GUIFactory):
    def create_button(self): return WindowsButton()
    def create_checkbox(self): return WindowsCheckbox()

class MacFactory(GUIFactory):
    def create_button(self): return MacButton()
    def create_checkbox(self): return MacCheckbox()

def render_ui(factory: GUIFactory):
    print(factory.create_button().render())
    print(factory.create_checkbox().render())

render_ui(WindowsFactory())  # Windows-style button / Windows-style checkbox
render_ui(MacFactory())      # Mac-style button / Mac-style checkbox
```

(Verified — output matches exactly.) The key guarantee: a `WindowsFactory` can never accidentally hand you a `MacCheckbox` paired with a `WindowsButton` — the whole *family* stays consistent. Factory Method is about **one product**; Abstract Factory is about **a matched set of products.** A useful way to remember it: Abstract Factory is often literally implemented *using* several Factory Methods internally, one per product it builds.

	Creates	Mechanism	Typical use
Simple Factory	One product, decided by a parameter	if/elif in one function	Quick and simple, but not open/closed
Factory Method	One product	Subclass overrides one method	Let subclasses pick the exact type
Abstract Factory	A *family* of related products	One interface, several factory methods	Ensure related objects are always used together

7. Python-Specific Ways to Implement This (beyond classic subclassing)

Python's flexibility means you rarely need the full class-hierarchy version from Section 5 for simple cases. Knowing these shows real Python maturity in an interview, not just textbook memorization.

7.1 Registry-based factory (fixes Simple Factory's Open/Closed problem, Pythonically)

Instead of an if/elif chain, classes register themselves in a dictionary — usually via a decorator:

```python
```

```
class TransportRegistry:
    _registry = {}

    @classmethod
    def register(cls, kind):
        def decorator(transport_cls):
            cls._registry[kind] = transport_cls
            return transport_cls
        return decorator

    @classmethod
    def create(cls, kind, *args, **kwargs):
        if kind not in cls._registry:
            raise ValueError(f"Unknown transport kind: {kind}")
        return cls._registry[kind](*args, **kwargs)


@TransportRegistry.register("road")
class Truck(Transport):
    def deliver(self):
        return "Delivering by land in a box"

@TransportRegistry.register("sea")
class Ship(Transport):
    def deliver(self):
        return "Delivering by sea in a container"


t = TransportRegistry.create("road")
print(t.deliver())  # Delivering by land in a box

TransportRegistry.create("air")
# ValueError: Unknown transport kind: air
```

(Verified — both the successful creation and the `ValueError` for an unknown kind.) This gives you the *convenience* of a Simple Factory (one call, pick by string) while keeping Open/Closed: adding `Plane` means adding a decorated class, and `TransportRegistry` itself never needs to change. Watch out for one subtle bug: if two classes accidentally register under the same key, the second one silently overwrites the first — worth guarding against in production code with an explicit check.

7.2 `@classmethod` as an alternate constructor ("named constructor")

This is Python's own idiomatic take on Factory Method, used constantly in the standard library — `dict.fromkeys()`, `datetime.fromtimestamp()`, `datetime.now()`.

The important detail: `cls` inside a classmethod refers to *whichever class it was actually called on* — including subclasses:

```python
class StuffedCrustPizza(Pizza):
    def __init__(self, ingredients):
        super().__init__(ingredients)
        self.crust = "stuffed"

p1 = Pizza.margherita()
p2 = StuffedCrustPizza.margherita()

print(type(p1).__name__)  # Pizza
print(type(p2).__name__)  # StuffedCrustPizza — cls automatically picked up the
```

(Verified.) `StuffedCrustPizza.margherita()` correctly returns a `StuffedCrustPizza`, not a `Pizza` — because `cls(...)` inside the classmethod is bound to whatever class the method was called through. This is Factory Method's spirit (defer to the calling context which concrete class gets built) without any of the subclassing ceremony from Section 5. Great answer to "have you seen this pattern in real Python code?"

8. Real-World Examples You Can Quote

- `dict.fromkeys(iterable)`, `datetime.fromtimestamp()`, `datetime.now()` — classmethod-based alternate constructors (Section 7.2).
- `csv.reader()` / `csv.writer()` — return different reader/writer objects depending on the dialect you specify.
- Django's form field system — `forms.CharField()`, `forms.IntegerField()`, etc. all conform to a common `Field` interface, created through a consistent pattern.
- Plugin/driver systems in larger frameworks, where the exact implementation class is chosen at runtime based on configuration (database driver, cloud provider SDK, file

parser for a given format).

9. Pros and Cons

Pros

- Avoids tight coupling between calling code and concrete product classes.
- Follows the Open/Closed Principle — new product types don't require touching existing code.
- Centralizes creation logic instead of scattering `SomeClass()` calls everywhere.
- Enables real polymorphism — calling code works purely against the abstract Product interface.

Cons

- The classic subclassing version can lead to "class explosion" — a new subclass for every product type, which is a lot of ceremony for simple cases.
- Adds a layer of abstraction that can be overkill if you genuinely only ever have one product type.
- In Python specifically, reaching straight for a multi-class hierarchy when a registry dict or a `@classmethod` would do the same job in fewer lines is a common over-engineering mistake — know when the lighter-weight version (Section 7) is the better real-world answer.

10. When to Use It (and When Not To)

Use it when:

- A class genuinely can't know in advance which exact class of object it needs to create (plugin systems, format-specific parsers, drivers).
- You want subclasses to be able to specify the objects they create, without rewriting shared logic (like `plan_delivery` above).
- You want the "which concrete class" decision centralized in one place instead of scattered through the codebase.

Be cautious when:

- You only have one product type and no real prospect of more — a plain constructor is simpler and clearer.
- In Python, before reaching for a full Creator/ConcreteCreator class hierarchy, ask whether a registry dictionary (7.1) or a `@classmethod` (7.2) already solves the actual problem with less code.

11. Factory Method vs. Abstract Factory vs. Builder — quick differentiation

Interviewers often ask you to distinguish these three "creational" patterns in one breath:

- **Factory Method** — one method decides *which class* to instantiate for a single product.
- **Abstract Factory** — one object bundling *several* factory methods, producing a matched *family* of related products (Section 6).
- **Builder** — not about deciding *which class*, but about constructing one *complex* object step by step, when that object has many optional parts or a multi-step assembly process (e.g. building an HTML report piece by piece, or a complex HTTP request object). Same final class every time, different *configuration*.

12. Interview Question Bank

Q1: What problem does Factory Method solve? It removes the direct coupling between calling code and concrete classes by deferring the "which class to build" decision to an overridable method — new product types can be added without touching existing code (Open/Closed Principle).

Q2: What's the difference between Simple Factory, Factory Method, and Abstract Factory? Simple Factory is one function with if/elif logic (not officially a GoF pattern, and not open/closed). Factory Method uses subclassing/polymorphism to let subclasses decide the concrete type of a single product. Abstract Factory bundles multiple factory methods to produce a consistent *family* of related products (Section 6).

Q3: Why use `ABC` and `@abstractmethod` for the Creator? It makes Python refuse to instantiate any subclass that forgot to override the factory method, failing loudly and immediately with a clear `TypeError` instead of failing silently or much later (verified in Section 5).

Q4: How would you implement Factory Method in idiomatic Python, not just the textbook OOP way? Mention the registry/decorator-based factory (7.1) and `@classmethod` alternate constructors (7.2) — both achieve the same "defer the creation decision" goal with less ceremony than a full Creator/ConcreteCreator class hierarchy.

Q5: What real examples of Factory Method exist in Python's standard library? `dict.fromkeys()`, `datetime.fromtimestamp()`, `datetime.now()` — classmethod-based named constructors.

Q6: What SOLID principle is Factory Method the textbook example for? The Open/Closed Principle — open for extension (add new subclasses/registrations), closed for modification (existing calling code never changes).

Q7: What happens if two classes register under the same key in a registry-based factory? The second registration silently overwrites the first in the dictionary — a real bug risk worth explicitly guarding against (e.g. raising an error on duplicate registration).

Q8: In a classmethod-based alternate constructor, why use `cls(...)` instead of hard-coding the class name? `cls` is bound to whatever class the method was actually called on, so subclasses automatically get correctly-typed instances back (verified with `StuffedCrustPizza.margherita()` returning a `StuffedCrustPizza`, not a `Pizza`) — hard-coding the class name would break that for every subclass.

Q9: When would you avoid Factory Method in Python? When there's only one product type with no real prospect of more — plain constructors are simpler — or when a registry dict / classmethod solves the same problem with far less boilerplate than a full class hierarchy.

Q10: How is Factory Method different from the Builder pattern? Factory Method decides *which class* to instantiate; Builder constructs *one* complex object step by step when it has many optional parts, without necessarily varying the final class at all (Section 11).

13. One-Page Cheat Sheet

- **Definition**: a method whose job is to decide which class to instantiate, so calling code never calls a concrete constructor directly.
- **Structure**: Creator (declares the factory method + business logic) → ConcreteCreator (overrides it) → Product interface → ConcreteProduct.
- **Solves**: tight coupling between calling code and concrete classes; violates-Open/Closed if you don't use it.
- **Simple Factory ≠ Factory Method**: if/elif in one function is Simple Factory (not a real GoF pattern, not open/closed); overriding a method in subclasses is the real Factory Method.
- **Abstract Factory ≠ Factory Method**: Abstract Factory bundles several factory methods to build a *family* of related, must-match-together products.
- **Gotcha**: forgetting to override the factory method with `ABC` + `@abstractmethod` raises `TypeError` immediately — a feature, not a bug.
- **Pythonic alternatives**: a registry dict + decorator (fixes Simple Factory's Open/Closed problem), or `@classmethod` alternate constructors (Python's own idiomatic Factory Method, seen throughout the standard library).
- **Real examples**: `dict.fromkeys()`, `datetime.fromtimestamp()`, Django form fields.
- **Watch for**: class explosion with the full subclassing version; duplicate keys silently overwriting each other in a registry-based factory.

This builds directly on the metaclass and OOP fundamentals from your Singleton prep — a natural next step is combining the two: a registry-based Factory Method that hands out objects, where the registry itself is implemented as a Singleton so the whole app shares one consistent set of registered types.